Experiment 13 — Case Study on Low-Code Limitations (Gym Membership Application)

Aim

To study the limitations of low-code development through a case study of the Gym Membership Enrollment & Front Desk Management application built in Zoho Creator, and to identify when custom code or pro-code extensions are required.

Procedure

1. Open Zoho Creator and open the Gym Membership Enrollment application.
2. Open the Membership Enrollment Form in the form builder and review the available fields and field properties.
3. Open the Workflow builder and review the email notification triggered on successful form submission.
4. Review the nine reports and note where standard report types cannot meet advanced requirements.
5. Open Share/Permissions and review the permission set for each module and the field-level permission options.
6. Open the Billing page and note the plan, user and application limits.
7. Relate each observation to a general low-code limitation and record the case study.

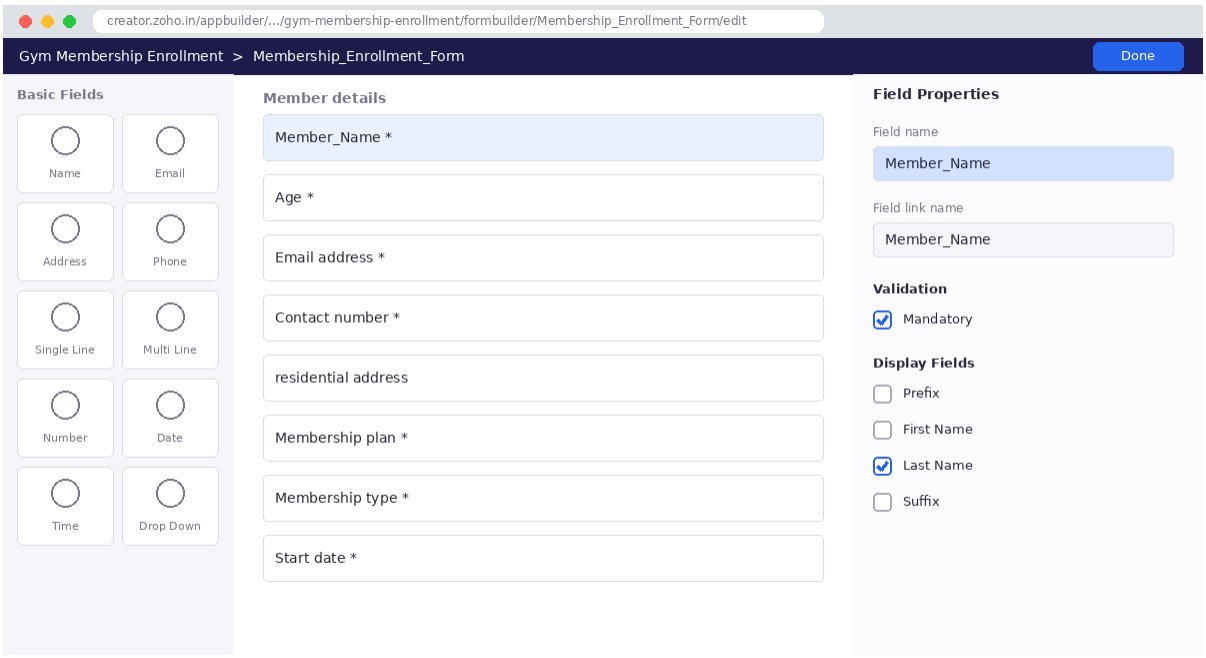
Introduction

Low-code development allows users to create applications using visual interfaces, drag-and-drop components, predefined templates and minimal programming. Zoho Creator helps build business applications quickly, automate workflows, manage data and create reports. However, limitations become noticeable when applications require complex logic, advanced customization, high performance, specialized integrations or strict security.

1. Limited Customization

Developers mainly use the components and features provided by the platform. Although scripting and custom functions are supported, complete control over the application's internal structure is not available.

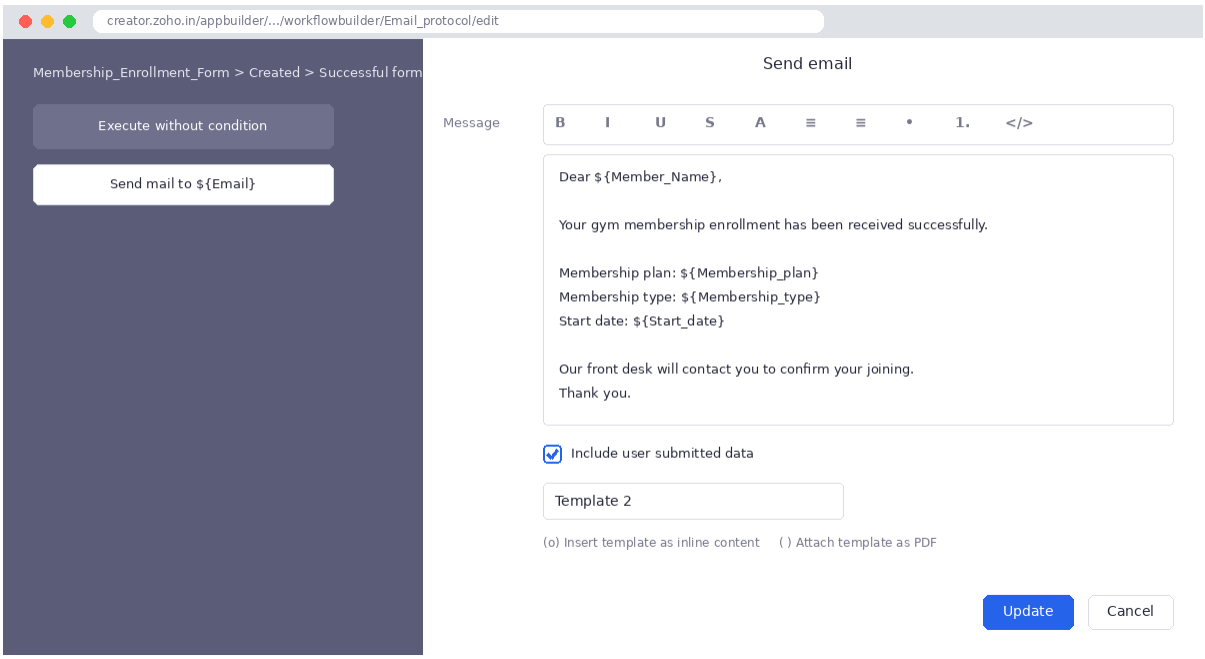
Example: *a standard enrollment form and reports are easy to build in Zoho Creator, but a highly customized member check-in kiosk interface or an interactive trainer-scheduling dashboard may need additional development or external tools.*



2. Complex Business Logic

Low-code platforms suit simple workflows, but advanced conditions, multiple dependent workflows, complicated calculations and exception handling may require custom scripting.

Example: *the gym may need to calculate fees by plan and membership type, apply corporate or renewal discounts, handle freeze requests, update payment status and send reminders. When several workflows depend on one another, debugging and maintenance become difficult.*



3. Performance and Scalability

As records and users increase, performance may decrease if forms, reports, workflows or queries are not optimized. Platforms also impose limits on records, API calls and workflow executions.

Example: *a gym chain with thousands of members may see slower report generation if every report fetches unnecessary fields or processes all records instead of using filters.*

4. Limited Integration with External Systems

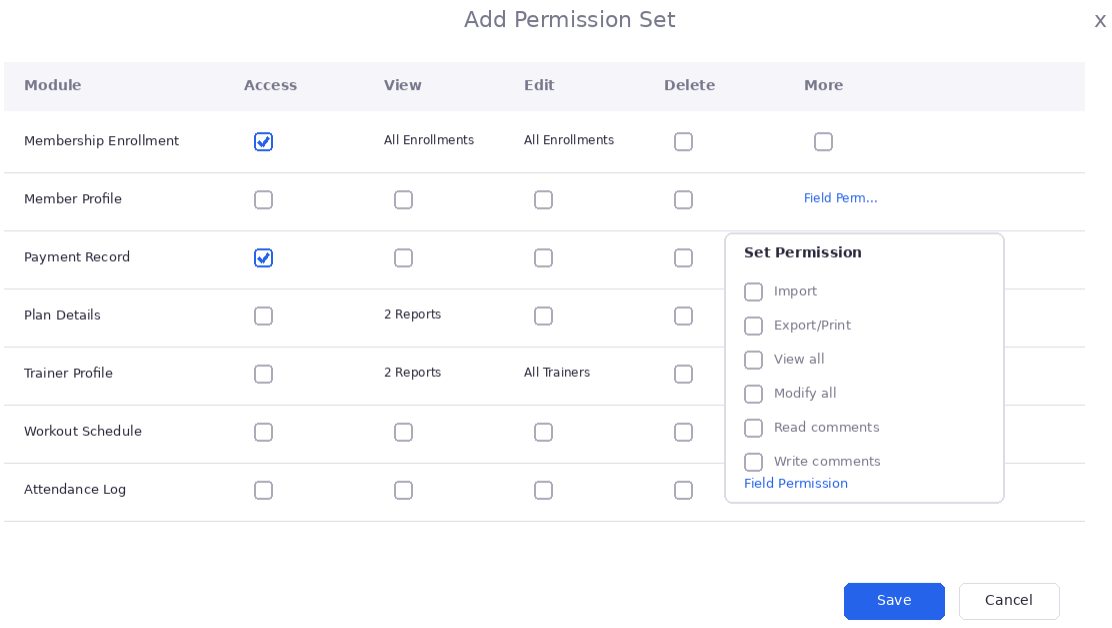
Businesses often connect applications to payment gateways, accounting software and third-party services. Some systems have no ready-made connector, so APIs, custom functions or middleware may be required.

Example: *connecting the enrollment app to a biometric door-access system or a UPI payment gateway may need custom API integration.*

5. Security and Data Privacy

Permissions, access controls and data-sharing settings are provided, but developers must configure them correctly. Incorrect settings can expose confidential data.

Example: *members' phone numbers, email addresses, residential addresses and payment details should not be visible to unrelated users. Trainers should see only the records they need.*



6. Vendor Lock-In

Forms, workflows, database structures and scripts use platform-specific features that cannot be transferred directly. Exporting data alone does not transfer the application's functionality.

Example: *moving the Zoho Creator gym application to a custom Python or Java application would require rebuilding its forms, workflows, permissions and reports.*

7. Limited Control over Source Code

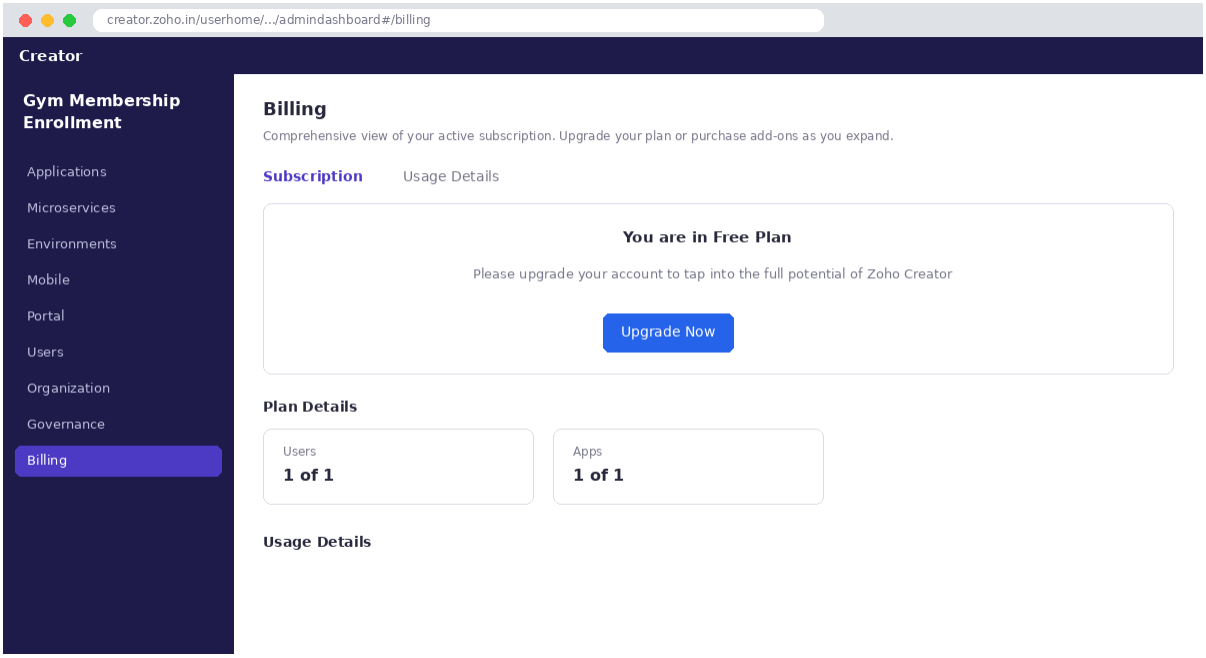
Low-code platforms abstract architecture, libraries and deployment. This can limit specialized algorithms, unsupported libraries and optimization of internal processing.

Example: *a custom workout-recommendation algorithm cannot be built directly inside the platform and needs a pro-code extension.*

8. Platform Dependency and Pricing

Applications depend on the platform's availability, features, pricing and policies. Some capabilities exist only in particular plans, so user limits, storage and automation quotas must be evaluated.

Example: *the free plan limits the number of users and apps, and expired trials restrict editing, as seen in the gym application.*



9. Testing and Maintenance Challenges

Low-code does not remove the need for testing, documentation, version management, backups and regression testing. A small change in one workflow can affect others.

Example: *changing the membership-type workflow may unintentionally affect renewal reminders or the Kanban view if dependencies are not tested.*

10. Suitability for Specialized Applications

Applications needing intensive real-time processing, advanced graphics, specialized hardware access or complete infrastructure control are better suited to traditional programming or a hybrid approach.

Example: *enrollment, reports and notifications suit low-code well, while live equipment-sensor tracking or AI-based fitness prediction would need custom code.*

Result

The case study of the Gym Membership Enrollment & Front Desk Management application shows that low-code tools are effective for enrollments, member records, reports and notifications, but have limitations in customization, complex logic, performance, integration, security, portability, source-code control, cost and maintenance. Advanced needs can be met with custom scripting, external integrations or pro-code extensions, so a combination of low-code and traditional programming gives the best flexibility.